# Final presentation script / 答辩演讲稿（16 min）

Slides: `slides.pdf`（17 页）。

- **Format（课程要求）**：4 人组讲 16 分钟，之后有 5–6 分钟提问。要求覆盖：动机和大背景、问题（配示例图）、我们的方法、解决的挑战和关键思路、可视化或演示、结论和洞见（已解决的挑战、新出现的挑战）。
- **怎么读这份稿**：每页先写英文讲稿（上台讲的），下面是中文对照（帮助理解和记忆，不用讲）。英文约 2,300 词，按正常语速大约 15.5 分钟。
- **讲者分工**：下表只是建议，4 段各约 4 分钟，可以自行调整。

| Part | Slides | Time | Speaker |
|---|---|---|---|
| A. Motivation, problem, approach | 1–6 | 4:00 | ? |
| B. Results, bottlenecks 1–2 | 7–9 | 4:00 | ? |
| C. Bottleneck 3, network, 10 Mbit/s, demo | 10–13 | 4:00 | ? |
| D. PP and RK3588, challenges, conclusions | 14–17 | 4:00 | ? |

**关于 demo（第 13 页）**：建议提前录一段 30–60 秒的屏幕录像（集群上运行 `scripts/chat.sh`），现场设备或网络出问题时就放录像。如果现场能连上集群，也可以直接演示。

---

## Part A — Motivation, problem, approach (4:00)

### Slide 1 — Title (0:20)

Hi everyone, we are group 7. Our project is about running a large language model on a cluster of small edge devices, in our case Raspberry Pis.

> 大家好，我们是第 7 组。我们的项目是在一组小型边缘设备上运行大语言模型，我们用的是树莓派。

### Slide 2 — Outline (0:15)

We'll start with the background and the problem, then show our results and the bottlenecks we found, and end with our conclusions.

> 我们先讲背景和问题，然后展示结果和找到的瓶颈，最后是结论。

### Slide 3 — Why run an LLM on edge devices? (1:00)

More and more cyber-physical systems, like robots, cars, smart homes and factories, want to use language models close to their sensors.
Today most LLMs run in the cloud. That means the data leaves the device, every request adds network latency, and nothing works without a connection.
A single Raspberry Pi can run a small model locally, but it is slow.
So our question was: if we connect several cheap boards, can they run one model together and be faster?

We had four research questions: how speed scales with the number of nodes, whether pipeline or tensor parallelism is better, what the bottleneck is, and whether we can fix it without losing accuracy.
For all experiments we used Llama 3.2 with 3 billion parameters, quantized to 4 bits, so the model file is about 1.9 GB.

> 越来越多的信息物理系统，比如机器人、汽车、智能家居和工厂，希望在靠近传感器的地方使用语言模型。
> 现在大多数大模型跑在云端：数据要离开设备，每次请求都有网络延迟，断网就用不了。
> 一块树莓派可以在本地跑小模型，但很慢。
> 所以我们的问题是：把几块便宜的板子连起来，能不能一起跑一个模型，并且更快?
> 我们有四个研究问题：速度怎样随节点数变化；流水线并行和张量并行哪个更好；瓶颈在哪里；能否在不损失准确率的前提下解决。
> 所有实验都用 Llama 3.2 的 30 亿参数版本，4 比特量化，模型文件约 1.9 GB。

### Slide 4 — Our cluster (0:50)

This is our actual setup in the lab. We have seven Raspberry Pis: the keyboards are Pi 400s, and the small boxes are Pi 4Bs. There is also one faster board with an RK3588 chip, the grey box at the back. Everything is connected to one Gigabit switch.
The table shows one important number: memory bandwidth. A Pi reads memory at about 4 GB per second. To generate one token, the model has to read all 1.9 GB of weights, so one Pi gets about 1.8 tokens per second. The RK3588 has five times the bandwidth and is about seven times faster.

> 这是我们在实验室的真实设备。7 台树莓派：键盘形状的是 Pi 400，小盒子是 Pi 4B。还有一块更快的 RK3588 板子，就是后面那个灰色盒子。所有设备连在同一台千兆交换机上。
> 表里有一个关键数字：内存带宽。树莓派每秒能读约 4 GB 内存。每生成一个 token 都要把 1.9 GB 权重读一遍，所以一台 Pi 每秒约 1.8 个 token。RK3588 的带宽是它的 5 倍，速度约快 7 倍。

### Slide 5 — The problem: how do you split a model over devices? (0:55)

The core problem is how to split one model across several devices. There are two basic ways.
With pipeline parallelism, at the top, each device gets a block of layers. A token goes through Pi 1, then Pi 2, and so on. It's simple and needs little communication, but only one device is working at any time.
With tensor parallelism, at the bottom, every layer is split across all devices, so they all work at the same time. The price is that after each part of a layer, the devices must add up their partial results. This is called an allreduce, and for our model it happens 56 times for every token.
So tensor parallelism trades computation for communication, and the question is which one wins on real hardware.

> 核心问题是怎样把一个模型拆到多台设备上，基本有两种办法。
> 上面是流水线并行：每台设备负责连续的几层，token 依次经过 Pi 1、Pi 2……这种方式简单、通信少，但任何时刻只有一台设备在算。
> 下面是张量并行：每一层都拆给所有设备，大家同时算。代价是每算完一部分，设备之间要把部分结果加起来，这叫 allreduce。对我们的模型，每生成一个 token 要做 56 次。
> 所以张量并行是用通信换计算，问题是在真实硬件上哪一方胜出。

### Slide 6 — Our approach (0:40)

We started with Distributed Llama as a baseline on one, two and four nodes. For the main experiments we used llama.cpp, because its RPC backend supports both pipeline and tensor parallelism on up to eight devices.
To find out *why* something was slow, we measured two things: a small monitor on every device that records CPU, memory and network every half second, and a timing log inside llama.cpp that shows how much time each node spends computing, synchronising or just waiting.

> 我们先用 Distributed Llama 在 1、2、4 个节点上跑基线。主要实验用 llama.cpp，因为它的 RPC 后端在最多 8 台设备上同时支持流水线并行和张量并行。
> 为了搞清楚慢的原因，我们测两样东西：每台设备上一个小监控程序，每半秒记录 CPU、内存和网络；以及 llama.cpp 内部的计时日志，记录每个节点花多少时间在计算、同步和空等上。

---

## Part B — Results, bottlenecks 1–2 (4:00)

### Slide 7 — Decode speed from 1 to 8 nodes (1:20)

This is our main result: decode speed, so how many tokens per second we generate, from one to eight nodes.
Pipeline parallelism is never faster than a single Pi. That's expected, because only one device is working at a time.
Stock tensor parallelism actually gets *slower* when we add nodes. On eight nodes it only reaches 0.41 tokens per second, more than four times slower than one Pi.
After our changes we get 6.34 tokens per second on eight nodes. That's 3.4 times faster than one Pi, and 15 times faster than stock llama.cpp.
So what was wrong with the stock version? We found three bottlenecks, and we'll go through them one by one.

> 这是我们的主要结果：1 到 8 个节点时的 decode 速度，也就是每秒生成多少个 token。
> 流水线并行从来没有比单台 Pi 快，这在意料之中，因为同一时刻只有一台设备在算。
> 原版张量并行在加节点后反而变慢：8 个节点只有每秒 0.41 个 token，比一台 Pi 慢 4 倍多。
> 用我们修改后的版本，8 个节点达到每秒 6.34 个 token，是单台 Pi 的 3.4 倍，原版的 15 倍。
> 原版问题出在哪？我们找到了三个瓶颈，下面逐个讲。

### Slide 8 — Bottleneck 1: all traffic goes through the head (1:20)

Remember the allreduce from slide 5: after each part of a layer, the nodes need to add up their partial results.
Stock llama.cpp only does this directly between nodes when there are exactly two. With more nodes, the head downloads all the partial results, adds them up and sends them back to everyone.
On four nodes, the head sends about 50 MB for every single token, as you can see on the left.
Our key idea was to let the nodes exchange data directly with each other, using a method called recursive doubling. Each node only talks to one partner per step, and it takes log-two-of-N steps. On four nodes, that took us from 1.03 to almost 3 tokens per second.

> 回想第 5 页的 allreduce：每算完一部分，各节点要把部分结果加起来。
> 原版 llama.cpp 只有在正好 2 个节点时才让节点直接交换。超过 2 个节点，就由主控把所有部分结果下载下来，加好再发回给每个节点。
> 左图可以看到，4 个节点时主控每生成一个 token 要发送约 50 MB。
> 我们的关键思路是让节点之间直接交换数据，方法叫递归倍增：每一步每个节点只和一个伙伴通信，一共 $\log_2 N$ 步。4 个节点从每秒 1.03 个 token 提升到接近 3 个。

### Slide 9 — Bottleneck 2: the output layer is not split (1:20)

Then we looked at the timing log. Each bar is one node. Even after the first fix, every node spent 80 to 85 milliseconds per token on the very last layer, the output layer, no matter how many nodes we had.
The reason is that Llama 3.2 shares this layer with the input embedding. llama.cpp then copies the whole layer to every node instead of splitting it.
Our fix was to store a separate copy of that layer in the model file. The output is exactly the same, and we checked that the perplexity does not change. Now the layer gets split like all the others, and four nodes went from 3 to 4.85 tokens per second, about 60 percent faster.

> 接着我们看计时日志，每根柱子是一个节点。即使做了第一个修改，每个节点每个 token 仍要在最后一层，也就是输出层上花 80 到 85 毫秒，而且跟节点数无关。
> 原因是 Llama 3.2 的输出层和输入嵌入层共用同一份权重，于是 llama.cpp 把整层复制到每个节点，而不是拆开。
> 我们的办法是在模型文件里给这一层单独存一份。输出完全一样，我们也检查过困惑度（perplexity）没有变化。这样这一层就和其他层一样被拆分，4 个节点从每秒 3 个 token 提升到 4.85 个，快了约 60%。

---

## Part C — Bottleneck 3, network, 10 Mbit/s, demo (4:00)

### Slide 10 — Bottleneck 3: the head's CPU at 8 nodes (1:10)

When we went from four to eight nodes, the speed suddenly dropped again, to 1.6 tokens per second. Our resource monitor showed why: the head's CPU was at 94 percent, while the workers were almost idle at 15 percent. The timing log agreed: of 631 milliseconds per token, the workers spent 470 just waiting for the head.
We attached gdb to the head and found that llama.cpp runs one thread per remote node, and these threads busy-wait: they keep spinning even when there's nothing to do. With eight nodes, that's eight spinning threads on a four-core Pi, and the threads doing real work don't get the CPU.
We added an option that lets these threads sleep instead. That took eight nodes from 1.62 to 6.34 tokens per second, almost four times faster.

> 从 4 个节点扩到 8 个时，速度又突然掉到每秒 1.6 个 token。
> 资源监控显示了原因：主控 CPU 占用 94%，worker 只有 15%，几乎闲着。计时日志也对得上：每个 token 631 毫秒里，worker 有 470 毫秒在等主控。
> 我们用 gdb 附加到主控进程，发现 llama.cpp 给每个远程节点开一个线程，这些线程是忙等的：没事做也在空转。8 个节点就是 8 个空转线程挤在 4 核的 Pi 上，真正干活的线程抢不到 CPU。
> 我们加了一个选项让这些线程改为休眠等待。8 个节点从每秒 1.62 个 token 提升到 6.34 个，快了将近 4 倍。

### Slide 11 — Is the network the bottleneck? (0:50)

A natural guess is that the network is the problem. We checked this with the monitor.
On Gigabit Ethernet, each worker used at most 233 out of 940 megabits per second, so about a quarter of the link.
And the strongest argument: when we fixed bottleneck 3, the network traffic stayed exactly the same, but the speed went up almost four times. So the network was not what limited us. For a single Pi, the limit is memory bandwidth, as we saw earlier.

> 很自然会猜是网络的问题，我们用监控数据验证了一下。
> 在千兆以太网上，每台 worker 最多只用了 940 Mbit/s 里的 233，大约四分之一。
> 最有力的证据是：修好第三个瓶颈后，网络流量完全没变，速度却提高了将近 4 倍。所以限制我们的不是网络。单台 Pi 的限制是内存带宽，前面已经讲过。

### Slide 12 — What if the network is slow? (1:00)

But edge devices don't always have a fast network. So we limited all eight devices to 10 megabits per second and compared stock llama.cpp with our version.
The stock version pushes 200 MB per token through the head. The link is completely full, and it generates about one token every 100 seconds.
With our direct allreduce, the head only handles 5.3 MB per token, and we get 0.2 to 0.26 tokens per second.
That's 20 to 26 times faster, compared with 15 times on Gigabit. So reducing traffic matters even more when the network is slow.

> 但边缘设备的网络不一定快。所以我们把 8 台设备都限速到 10 Mbit/s，对比原版和我们的版本。
> 原版每个 token 要经过主控传 200 MB，链路完全跑满，大约 100 秒才出一个 token。
> 用我们的直接 allreduce，主控每个 token 只处理 5.3 MB，速度是每秒 0.2 到 0.26 个 token。
> 快了 20 到 26 倍，而在千兆网络下是 15 倍。所以网络越慢，减少流量就越重要。

### Slide 13 — Demo (1:00)

Let's see what this looks like in practice. Here we ask the eight-node cluster a question.
*(play the video or run `scripts/chat.sh "What is a Raspberry Pi?"`)*
All eight devices work on every token. With our changes the text comes out at about six tokens per second, which is fast enough to read along. With stock llama.cpp, you would wait about two and a half seconds for every token.

> 我们来看实际效果，这里向 8 节点集群提一个问题。
> （播放录像，或现场运行 `scripts/chat.sh "What is a Raspberry Pi?"`）
> 每个 token 都由 8 台设备一起计算。用我们修改后的版本，每秒大约出 6 个 token，基本能跟着读；用原版，每个 token 要等约 2.5 秒。

---

## Part D — PP and RK3588, challenges, conclusions (4:00)

### Slide 14 — Pipeline parallelism and a faster board (1:15)

Two shorter results.
On the left, pipeline parallelism. For a single request it can't be faster than one board. With several requests at the same time, the stages can overlap, and we got 29 percent more throughput. So the real use of pipeline parallelism is to fit a model that is too big for one board, not to make it faster.
On the right, the RK3588. Just compiling llama.cpp for its CPU doubled its speed, from 5.8 to 12.3 tokens per second. In a mixed cluster, giving the fast board a bigger share helps a lot: on two nodes we went from 3.2 to 7 tokens per second. But honestly, for this model, the RK3588 alone is faster than our whole Pi cluster.

> 两个较短的结果。
> 左边是流水线并行。单个请求时它不可能比一块板子快。同时有多个请求时各段可以重叠，吞吐量提高了 29%。所以流水线并行真正的用处是装下单台放不下的模型，而不是提速。
> 右边是 RK3588。只是针对它的 CPU 重新编译 llama.cpp，速度就翻了一倍，从每秒 5.8 个 token 到 12.3 个。在混合集群里给快的板子多分一些很有用：2 个节点从每秒 3.2 个 token 提升到 7 个。但老实说，对这个模型，RK3588 一块板子就比我们整个树莓派集群快。

### Slide 15 — Practical challenges in the lab (0:50)

Not everything went smoothly. The switch was reset and lost its DHCP settings, so we gave every board a fixed IP. One Pi kept rebooting in the middle of a test, and the cause turned out to be a weak power supply. The Pis have no real-time clock, so we had to correct clock offsets before we could compare logs from different devices. And our first monitoring script was a shell script that slowed decoding down by 5 percent, so we rewrote it as a small C program.

> 过程并不都顺利。交换机被重置后丢了 DHCP 配置，我们就给每块板子设了固定 IP。有一台 Pi 在测试中途反复重启，最后发现是电源供电不足。树莓派没有实时时钟，所以比较不同设备的日志之前，要先修正时钟偏差。我们最初的监控是 shell 脚本，让 decode 慢了 5%，于是改写成一个小的 C 程序。

### Slide 16 — Conclusions and insights (1:45)

To sum up and answer our research questions:
Tensor parallelism scales to 3.4 times on eight Pis. Pipeline parallelism does not speed up a single request.
Tensor parallelism needs much more communication, but on Gigabit Ethernet the network was not the limit. The real limits were things that don't get smaller when you add nodes: the head relaying all traffic, the unsplit output layer, and the head's CPU.
All three fixes are small changes, and none of them affects accuracy.
Some new challenges came up along the way. Every fix revealed the next bottleneck; we only found the busy-waiting problem when we went to eight nodes. Mixed devices need uneven splits, and our fast allreduce doesn't support that yet. And one modern board beats seven Pis for a 3B model, so clusters like ours really pay off for models that don't fit on a single board. That's where we would continue.

> 总结一下，回答我们的研究问题：
> 张量并行在 8 台 Pi 上能达到 3.4 倍；流水线并行不能加速单个请求。

> 张量并行需要多得多的通信，但在千兆以太网上网络不是瓶颈。真正的限制是那些不会随节点增加而变小的部分：主控中转全部流量、输出层没有拆分、主控 CPU。
> 三个修改都很小，而且都不影响准确率。
> 过程中也出现了新的挑战。每解决一个瓶颈，下一个就会冒出来；忙等问题是扩到 8 个节点才发现的。
> 混合设备需要不均匀的切分，我们的快速 allreduce 还不支持。另外，对 3B 模型，一块新板子就比 7 台 Pi 快，所以这样的集群真正有价值的场景是单台装不下的模型，这也是我们下一步想做的方向。

### Slide 17 — Thank you (0:10)

Thank you for listening. We're happy to take questions.

> 谢谢大家，欢迎提问。

---

## Possible questions / 可能的提问

**Q: Why not just use the RK3588?**
For this 3B model it is the better choice. The cluster becomes useful for models that don't fit on one board, and when only weak devices are available.

> 对 3B 模型确实用它更好。集群的价值在于运行单台装不下的模型，或者手上只有弱设备的情况。

**Q: Does untying the output layer change the model?**
No. It is an exact copy of the same weights. The output tokens are identical and the perplexity is the same.

> 不会。只是同一份权重的完整拷贝，输出 token 完全一样，困惑度也相同。

**Q: Why only powers of two?**
Recursive doubling pairs nodes up in each step. Uneven splits or node counts that are not a power of two fall back to the head relay. Supporting them is our main future work.

> 递归倍增每一步都要两两配对。不均匀切分或节点数不是 2 的幂时，会退回主控中转的方式。支持这些情况是我们主要的后续工作。

**Q: How reliable are the numbers?**
Each configuration was run one or two times with the monitor on, and the monitor costs at most 2 to 3 percent. The effects we report are much larger than that.

> 每个配置在开着监控的情况下跑了 1–2 次，监控开销最多 2–3%，我们报告的效果远大于这个误差。

**Q: Why does pipeline parallelism not help?**
With one request, the layers run in sequence, so only one device works at a time. Pipeline parallelism adds memory, not speed.

> 单个请求时各层是按顺序执行的，同一时刻只有一台设备在算。流水线并行增加的是内存容量，不是速度。